Manual técnico: FlotaControl

Instalación, arquitectura, configuración, pruebas y mantenimiento · Rosa Margarita Vivanco Vélez · UNEMI · octubre de 2026

1. Alcance

Este manual es para quien instale, opere o continúe el desarrollo de FlotaControl.

Repositorio: https://github.com/rvivancov-del/control_flota.

Cómo están escritos los comandos: se ejecutan desde la raíz del repositorio. Son de Git Bash en Windows, o de cualquier terminal en Linux o macOS.

2. Arquitectura

Estilo: microservicios de grano grueso, es decir, tres servicios más un API gateway. El frontend es una SPA.

Por qué: se eligió con una matriz de decisión ponderada. Los microservicios obtuvieron 3,95, frente a 3,60 del monolito modular y 3,35 del monolito en capas.

Contenedor	Tecnología	Puerto	Responsabilidad
`gateway`	Nginx	**8080** (publicado)	Punto único de entrada: `/api/auth`, `/api/fleet`, `/api/reports`; límite de tasa del login y del despacho
`auth-service`	Python 3.12 + FastAPI	8000 (interno)	Usuarios, roles, contraseñas con bcrypt, JWT, bloqueo por intentos
`fleet-core-service`	Python 3.12 + FastAPI + SQLAlchemy + Alembic	8000 (interno)	Vehículos, conductores, viajes, gastos, mantenimiento, costos fijos y de personal, despacho QR, agregaciones `/analytics/*`
`reporting-service`	Python 3.12 + FastAPI	8000 (interno)	Tablero, modelo de costos, AHP, reportes y exportación a Excel y PDF. **Sin base de datos:** consulta a fleet-core por HTTP con el token del usuario
`postgres`	PostgreSQL 16	5432 (interno, no publicado)	Esquemas `auth` y `fleet`, con un rol dueño por servicio (`infra/postgres/roles.sql`)
`frontend`	React 18 + TypeScript 5 + Vite 5	**5173** (publicado)	Panel web y portal móvil del conductor

Flujo de una petición:

- El navegador llama a `http://<host>:8080/api/...` (ver `frontend/src/services/apiClient.ts`).
- El gateway la enruta al servicio que corresponde (`infra/nginx/nginx.conf`).
- El servicio valida el JWT y el rol (`app/core/deps.py`).
- Responde con datos validados por los esquemas de Pydantic.

Diagramas: en `docs/diagramas/`: contenedores, despliegue, casos de uso, secuencias, estados, actividades y modelo entidad-relación.

3. Requisitos

Programa	Versión	Para qué
Docker Desktop (o Docker Engine con Compose v2)	4.x o superior	Ejecutar todo el sistema
Git	2.x	Clonar el repositorio
Python	3.12 o superior (opcional)	Scripts de `scripts/` y generadores
Node.js	18 o superior (recomendado)	Arranque con un solo comando (`npm run dev`) y compilar el frontend fuera de Docker
VS Code + extensión CodeTour (opcional)	—	Recorridos guiados del código (`.tours/`)

Para ejecutar el sistema no hace falta instalar Python ni PostgreSQL: todo corre en contenedores. Node.js solo se usa para el arranque con un solo comando; sin él, el sistema se levanta igual con `docker compose`.

4. Instalación y puesta en marcha

Arranque con un solo comando (recomendado). Desde la carpeta del repositorio, en CMD, PowerShell o la terminal de VS Code:

```
git clone https://github.com/rvivancov-del/control_flota.git
cd control_flota
npm run dev
npm run codigo
```

`npm run dev` ejecuta `scripts/dev.mjs`, que solo usa módulos nativos de Node, por lo que no requiere `npm install`. Hace lo siguiente:

1. Comprueba Docker. En Windows, abre Docker Desktop si está apagado y espera a que el motor responda.
2. Avisa si los puertos 5173 u 8080 están ocupados por otro programa.
3. Ejecuta `docker compose up --build -d`: las dependencias de Python y de Node se instalan dentro de las imágenes.
4. Espera a que PostgreSQL y los tres servicios estén «healthy» y, si el gateway arrancó antes que los servicios, lo reinicia.
5. Si la tabla `fleet.vehicles` está vacía, carga la base piloto ficticia (`scripts/seed/pilot_dataset.sql`) y verifica los conteos contra su manifiesto. Si ya hay datos, no modifica nada.
6. Muestra la dirección, el usuario inicial y abre el navegador (`--sin-navegador` lo evita).

`npm run codigo` abre el proyecto en VS Code con el espacio de trabajo `flotacontrol.code-workspace`. En VS Code, **Ctrl + Shift + B** ejecuta la tarea «Levantar FlotaControl».

Comando	Qué hace
`npm run dev`	Construye, levanta, carga los datos de prueba si la base está vacía y abre el navegador
`npm run dev:logs`	Igual que el anterior y después sigue los registros
`npm run estado`	`docker compose ps`
`npm run logs`	`docker compose logs -f`
`npm run stop`	Detiene los contenedores sin borrar datos
`npm test`	Pruebas unitarias de los tres servicios en contenedores desechables

`npm run codigo`	Abre el espacio de trabajo en VS Code

Ningún atajo ejecuta `docker compose down -v`, que borra el volumen de la base de datos.

Arranque manual (sin Node.js):

```
docker compose up --build -d
docker compose ps
```

Qué hacer después:

- **Comprobar los contenedores:** los 6 deben estar en «Up» o «healthy».
- **Abrir la aplicación:** http://localhost:5173.
- **Ingresar:** con el usuario inicial del entorno local, `admin` / `Admin1234`, que crea `auth-service/app/seed.py`.
- **Si aparece «No se pudo conectar con el servidor»:** `docker compose restart gateway`. El gateway resuelve las direcciones de los servicios al arrancar.

Migraciones: cada servicio con base de datos ejecuta `alembic upgrade head` al arrancar (ver su `Dockerfile`). auth-service además crea los roles y el administrador inicial.

Datos de prueba ficticios: 12 vehículos, 13 conductores, 640 viajes, 1.330 gastos y 96 mantenimientos.

```
docker compose exec -T postgres psql -U flota -d flota_db -v ON_ERROR_STOP=1 -q <
scripts/seed/pilot_dataset.sql
python scripts/seed_costos_ejemplo.py --db flota_db --viva     # costos fijos y de
personal ILUSTRATIVOS (opcional)
```

4.1 Descargar o actualizar a la última versión desde GitHub

A) Computadora nueva (todavía no tiene el proyecto)

```
git clone https://github.com/rvivancov-del/control_flota.git
cd control_flota
docker compose up --build -d
```

B) La computadora ya tiene la versión del Avance 2: actualizar al Avance 3

```
cd control_flota
git status
git stash
git fetch origin --tags
git checkout main
git pull origin main
docker compose down
docker compose up --build -d
docker compose restart gateway
docker compose ps
```

Qué hace cada comando:

- `cd control_flota`: entra a la carpeta del proyecto, o a la carpeta donde lo clonó.
- `git status`: muestra si hay cambios locales.

- `git stash`: solo hace falta si `git status` mostró archivos modificados. Guarda esos cambios aparte; se recuperan con `git stash pop`.
- `git fetch origin --tags`: descarga la versión nueva y las etiquetas de cada entrega.
- `git checkout main`: se ubica en la rama principal.
- `git pull origin main`: trae la última versión, la del Avance 3.
- `docker compose down`: detiene el sistema y **conserva** la base de datos.
- `docker compose up --build -d`: reconstruye las imágenes con el código nuevo.
- `docker compose restart gateway`: refresca las rutas del gateway hacia los servicios recién creados.
- `docker compose ps`: los 6 contenedores deben estar en «Up» o «healthy».

Comprobaciones:

- `git log --oneline -1` debe mostrar el último commit de GitHub, y `git describe --tags` debe mostrar `v2.0-avance3`.
- En http://localhost:5173 → **Mantenimiento**, la prioridad debe mostrar «seguridad (53 %), urgencia (42 %) y costo (5 %)».

Datos: del Avance 2 al Avance 3 no hay migraciones nuevas de la base. Los datos se conservan y no hace falta `docker compose down -v`.

Volver a una entrega anterior (solo para consultarla):

- `git checkout v1.1-avance2` muestra el Avance 2.
- `git checkout main` vuelve al Avance 3.
- Después de cambiar de versión, ejecute `docker compose up --build -d`.

Si `git pull` dice que hay conflictos: el proyecto se modificó a mano en esa computadora. Para descartar esos cambios y quedar exactamente igual que GitHub:

```
git fetch origin
git reset --hard origin/main
```

Cuidado: este comando borra los cambios locales que no se hayan guardado con `git stash`.

5. Configuración

Cada servicio lee su archivo `services/<servicio>/.env.example`. En producción se copia a `.env` o se pasan las variables de entorno.

Variable	Servicio	Valor de desarrollo	Descripción
`APP_ENV`	los tres	`development`	Fuera de `development`, el servicio **no arranca** con secretos por defecto (RNF-02)
`JWT_SECRET_KEY`	los tres	`changeme_local_only`	Clave para firmar el JWT. Debe ser **la misma** en los tres servicios
`JWT_EXPIRE_MINUTES`	auth	60	Duración de la sesión
`LOGIN_MAX_FAILED_ATTEMPTS` / `LOGIN_LOCKOUT_MINUTES`	auth	5 / 15	Bloqueo por intentos fallidos
`ADMIN_USERNAME` / `ADMIN_INITIAL_PASSWORD`	auth	admin / Admin1234	Administrador inicial; fuera de desarrollo se debe cambiar en el primer ingreso

POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_SCHEMA	auth y fleet	flota_auth / flota_fleet	Rol de base de datos propio de cada servicio
CORS_ORIGINS	los tres	http://localhost:5173	Orígenes del navegador autorizados, separados por comas
FLEET_SERVICE_BASE_URL	reporting	http://fleet-core-service:8000	Dirección interna de fleet-core
QR_TOKEN_TTL_DAYS	fleet	180	Vigencia de la credencial QR
MAINTENANCE_POLICY_PATH	fleet	—	Otra política de alertas (por defecto app/maintenance/maintenance_policy.json)
AHP_CONFIG_PATH	reporting	—	Otra configuración del AHP (por defecto app/dashboards/ahp_config.json)
UVICORN_WORKERS	los tres	2	Procesos por contenedor

Archivos de configuración del negocio. Cada uno declara su `estado` y su `procedencia`, y un valor inválido detiene el arranque.

- **`maintenance_policy.json` (fleet):** el umbral «próximo» es de **1000 km**, la mediana de los expertos.
- **`ahp_config.json` (reporting):** contiene las comparaciones de Saaty de los expertos, la criticidad, la dirección del costo y la curva de urgencia.
  - **Cómo se genera:** con `07_MATRICES_DE_ANALISIS/generadores/gen_ahp_config_expertos.py`, desde el libro de juicios.
  - **Validación:** si el CR es de 0,10 o más, el servicio no arranca.
- **`iva_config.json` (reporting) y `frontend/src/utils/tax.ts`:** tarifas de IVA por rubro (combustible y mantenimiento al 15 %; peajes al 0 %). `scripts/check_iva_rates.py` comprueba que coincidan.

Celular en la red local (despacho por QR):

- Abra el panel con la IP de la computadora (por ejemplo, `http://192.168.1.37:5173`), así el QR codifica esa dirección.
- Agregue ese origen a `CORS_ORIGINS` de los tres servicios (por ejemplo, `http://localhost:5173,http://192.168.1.37:5173`).
- Vuelva a crear los contenedores: `docker compose up -d`.

6. Estructura del repositorio

Ruta	Contenido
docker-compose.yml	Entorno local completo
docker-compose.prod.yml, .env.prod.example	Producción local con HTTPS: imágenes multietapa, sin usuario root
infra/nginx/	Configuración del gateway (desarrollo, producción y verificación)
infra/postgres/	Roles por servicio (roles.sql) e inicialización (init.sh)
services/<servicio>/app/main.py	Arranque de la API, CORS, routers y /health
services/<servicio>/app/core/	Configuración, base de datos, seguridad, dependencias de autenticación y roles

`services/<servicio>/app/models/`	Modelos ORM de SQLAlchemy
`services/<servicio>/app/<módulo>/`	`router.py` (endpoints) y `schemas.py` (validación con Pydantic) por módulo
`services/<servicio>/migrations/versions/`	Migraciones de Alembic
`services/<servicio>/tests/`	Pruebas unitarias y de integración (`test_*_smoke.py`)
`frontend/src/pages/`	Una página por ruta
`frontend/src/services/`	Clientes HTTP hacia el gateway (`apiClient`, `fleetApi`, `reportsApi`, `authApi`, `despachoApi`)
`frontend/src/context/`, `components/`, `charts/`, `types/`, `utils/`	Sesión, componentes comunes, gráficas, tipos y utilidades
`scripts/`	Base de prueba, informes de pruebas y cobertura, benchmark, carga, capturas y enlace de despacho (ver `scripts/README.md`)
`.tours/`, `flotacontrol.code-workspace`	Recorridos guiados del código y espacio de trabajo de VS Code
`.github/workflows/ci.yml`	Integración continua

7. Desarrollo

- **Aplicar un cambio de código:** reconstruya el servicio con `docker compose up --build -d <servicio>`. Si el navegador muestra 502, ejecute `docker compose restart gateway`.
- **Agregar un endpoint:** créelo en el `router.py` del módulo, con su esquema de Pydantic. Proteja el endpoint con `require_role(...)` de `app/core/deps.py` y agregue su prueba en `tests/`.
- **Nueva migración** (fleet-core o auth):
  - cree el archivo en `services/<servicio>/migrations/versions/`, con `revision` y `down_revision` encadenados a la última;
  - reconstruya el servicio, que la aplica solo al arrancar;
  - si se cambia el esquema, regenere el diagrama entidad-relación con `python scripts/gen_er_diagram.py`.
- **Nueva pantalla del frontend:**
  - agregue la página en `frontend/src/pages/` y su ruta en `App.tsx`;
  - si va en el menú, agréguela en `nav.ts`;
  - las llamadas van en el cliente que corresponda de `frontend/src/services/`.
- **Código compartido:** algunos archivos se copian a propósito entre los tres servicios. `python scripts/check_shared_code.py` comprueba que sigan idénticos.

8. Pruebas

Las pruebas no van dentro de las imágenes de producción. Se corren en un contenedor que monta la carpeta del servicio.

Unitarias (no necesitan el sistema levantado). La forma más corta es `npm test`, que construye las imágenes y corre las pruebas unitarias de los tres servicios, cada una en un contenedor desechable (837 pruebas). El equivalente manual, por servicio:

```
cd services/reporting-service
MSYS_NO_PATHCONV=1 docker run --rm --user root -v "$(pwd -W):/app" -w /app -e
APP_ENV=development \
  -e PYTHONPATH=/home/appuser/.local/lib/python3.12/site-packages sistema-control-
flota-reporting-service \
  sh -c "pip install -q -r requirements-test.txt httpx==0.27.2 && python -m pytest
tests -m 'not smoke' -q"
```

- En Linux o macOS se usa `$(pwd)` en lugar de `$(pwd -W)` y se omite `MSYS_NO_PATHCONV=1`.
- Cambie el nombre del servicio y de la imagen para auth o fleet-core.

Integración (contra una pila aislada, para no escribir en la base de trabajo):

```
python scripts/make_verif_nginx.py
docker compose -p flota_verif -f docker-compose.yml -f infra/docker-
compose.verif.yml up -d
docker compose -p flota_verif exec -T postgres psql -U flota -d flota_db -v
ON_ERROR_STOP=1 -q < scripts/seed/pilot_dataset.sql
```

Luego se corre el mismo `docker run` con `-e FLOTA_BASE_URL=http://host.docker.internal:18080/api` y sin `-m 'not smoke'`.

Resultado vigente: 910 pruebas, 909 aprobadas y 1 omitida por diseño. La omitida es la que agota el límite del gateway y solo corre con `RUN_RATE_LIMIT_TESTS=1`.

Cobertura de las pruebas unitarias: 80 %.

Integración continua: GitHub Actions corre las unitarias, construye las imágenes y levanta la pila completa en cada push.

Informes:

- `scripts/gen_informe_pruebas.py --xml-dir <carpeta> --salida <archivo>`;
- `scripts/gen_coverage_report.py`;
- `scripts/loadtest/` (k6) y `scripts/bench_queries.py`.

9. Operación

Tarea	Comando
Ver el estado	`docker compose ps`
Ver los registros de un servicio	`docker compose logs -f <servicio>`
Detener (conserva los datos)	`docker compose down`
Detener y **borrar** la base	`docker compose down -v`
Respaldo	`docker compose exec -T postgres pg_dump -U flota -d flota_db --clean --if-exists > respaldo.sql`
Restaurar	`docker compose exec -T postgres psql -U flota -d flota_db < respaldo.sql`
Producción local con HTTPS	`cp .env.prod.example .env.prod` (completar secretos), `python scripts/gen_dev_tls_cert.py`, `docker compose -f docker-compose.prod.yml up -d`
Enlace del portal del conductor para una demostración	`python scripts/enlace_despacho_demo.py`

10. Seguridad y datos personales

- **Contraseñas:** se guardan con bcrypt. La sesión usa un JWT firmado con el rol y una expiración de 60 minutos.
- **Autorización:** cada endpoint exige su rol. Sin token responde 401; con un rol no permitido, 403.
- **Base de datos:** cada servicio tiene un rol propio, sin permisos sobre el esquema del otro. La base no se publica fuera de Docker.
- **Gateway:** limita los intentos del login (10 por minuto) y del despacho (30 por minuto), y no registra la cadena de consulta de las URL.
- **Credencial QR:** se guarda como hash, vence a los 180 días y se puede revocar. Viaja en el fragmento de la URL y en una cabecera, no en los registros.
- **LOPDP:** los datos del conductor se pueden rectificar y suprimir; la supresión anonimiza de forma irreversible.

11. Solución de problemas

Síntoma	**Causa y solución**
«No se pudo conectar con el servidor», o 502 y 404 en `/api`	El gateway quedó con las direcciones viejas de los servicios: `docker compose restart gateway`
failed to connect to the docker API	Docker Desktop está cerrado: ábralo y espere a que diga *Engine running*
port is already allocated	Otro programa usa el 5173 o el 8080: ciérrelo o cambie el puerto publicado en `docker-compose.yml`
Un servicio queda en *Restarting*	`docker compose logs <servicio>`: suele ser una configuración inválida (`ahp_config.json` con CR ≥ 0,10, la política de alertas o un secreto por defecto fuera de desarrollo)
El QR no abre desde el celular	Mismo Wi-Fi, panel abierto con la IP de la computadora y esa IP agregada en `CORS_ORIGINS` (sección 5)
`docker compose exec <servicio> pytest` no encuentra pruebas	Las pruebas no se copian a la imagen: use el `docker run` con la carpeta montada (sección 8)